Convolutional Neural Networks (CNNs)

Architecture, Convolutions, and Backpropagation

Instructor: Prof. Dr. O. Olawale Awe

Motivation: Why Convolutions?

- Images have spatial structure: nearby pixels contain related information.
- Fully Connected (Dense) Networks ignore spatial locality.
- CNNs extract local patterns (edges, textures, shapes) automatically.
- Convolutions reduce parameters while preserving spatial information.

Key Idea of CNN

Convolution Operation

A filter slides across the image and computes:

$$(\mathbf{I} * \mathbf{K})(i,j) = \sum_m \sum_n I(i+m, j+n)\, K(m,n)$$

- Filters learn visual features.
- Multiple filters produce multiple feature maps.
- Deep layers build hierarchical representations.

What is a CNN?

A Convolutional Neural Network processes images by extracting spatial features:

- Local patterns (edges, corners, textures)
- Hierarchical structure (simple $\rightarrow$ complex)
- Parameter sharing across the image

Core layers:

- Convolution
- Activation (ReLU)
- Pooling
- Fully connected layers

How?

A convolution uses a small filter to detect patterns:

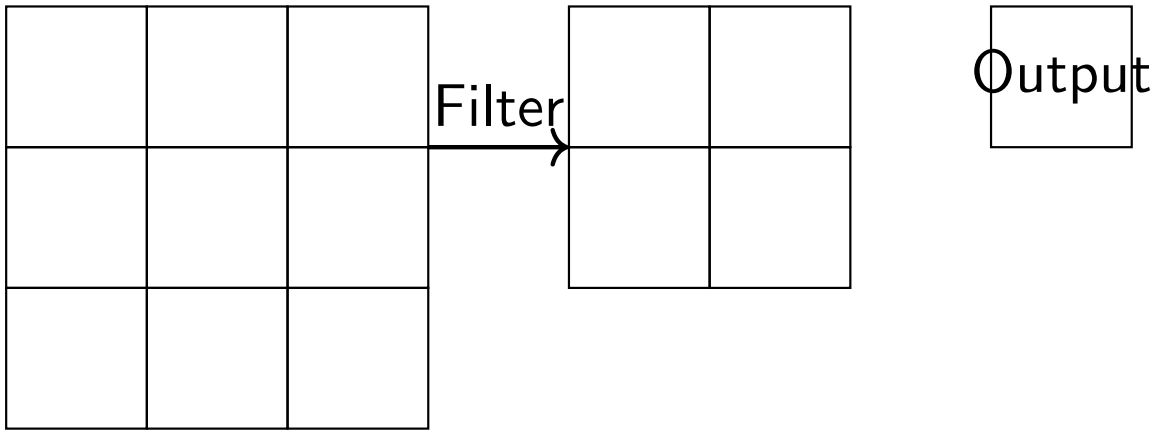
$$Y = X * W + b$$

- Filters slide across the image.
- Each filter learns a specific pattern.
- Produces a feature map.

Early filters:

- Detect edges, corners, gradients.

Diagram: Convolution



Numerical Example: Convolution

Image patch:

$$X = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}$$

Compute:

$$Y = (2)(1) + (1)(-1) + (0)(2) + (3)(0)$$

$$Y = 2 - 1 = 1$$

Interpretation: Filter detects differences between top-left and top-right.

ReLU Activation

After convolution:

$$Z = X * W + b$$

Activate:

$$A = \max(0, Z)$$

Why ReLU?

- Removes negative values
- Introduces non-linearity
- Avoids vanishing gradient (better than sigmoid/tanh)

Why Padding?

Without padding, output shrinks:

$$\text{Output size} = n - k + 1$$

With SAME padding:

$$\text{Output size} = n$$

Reasons:

- Keeps spatial resolution
- Preserves edges
- Allows deeper CNNs

What is Stride?

Stride s = number of pixels the filter moves.

$$\text{Output size} = \frac{n-k}{s} + 1$$

Effects:

- $s = 1$: detailed scanning
- $s = 2$: downsampling

Advantages of Padding and Stride

Padding (P):

- Preserves spatial size.
- Ensures edge pixels contribute to the output.

Stride (S):

- Controls how far the filter moves per step.
- Larger stride $\rightarrow$ smaller output map.

Pooling: Downsampling Features

- Reduces spatial dimensions.
- Controls overfitting.

Max Pooling Example

$$\text{MaxPool}(2 \times 2) = \max\{4, 2, 1, 3\}$$

Pooling (Downsampling)

Max pooling example:

$$\begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix} \Rightarrow \max = 3$$

Benefits:

- Reduces dimensionality
- Keeps strongest features
- Provides translation invariance

Pooling Diagram

0	3
2	1

$\xrightarrow{\text{Max Pool}}$

3

Flattening

Transforms 3D tensor:

$$H \times W \times C$$

into 1D vector.
Purpose:

- Feed into Dense layers
- Combine high-level features

Fully Connected Layers

Standard neural network layer:

$$y = Wh + b$$

Role:

- Integrate extracted features
- Perform classification or regression

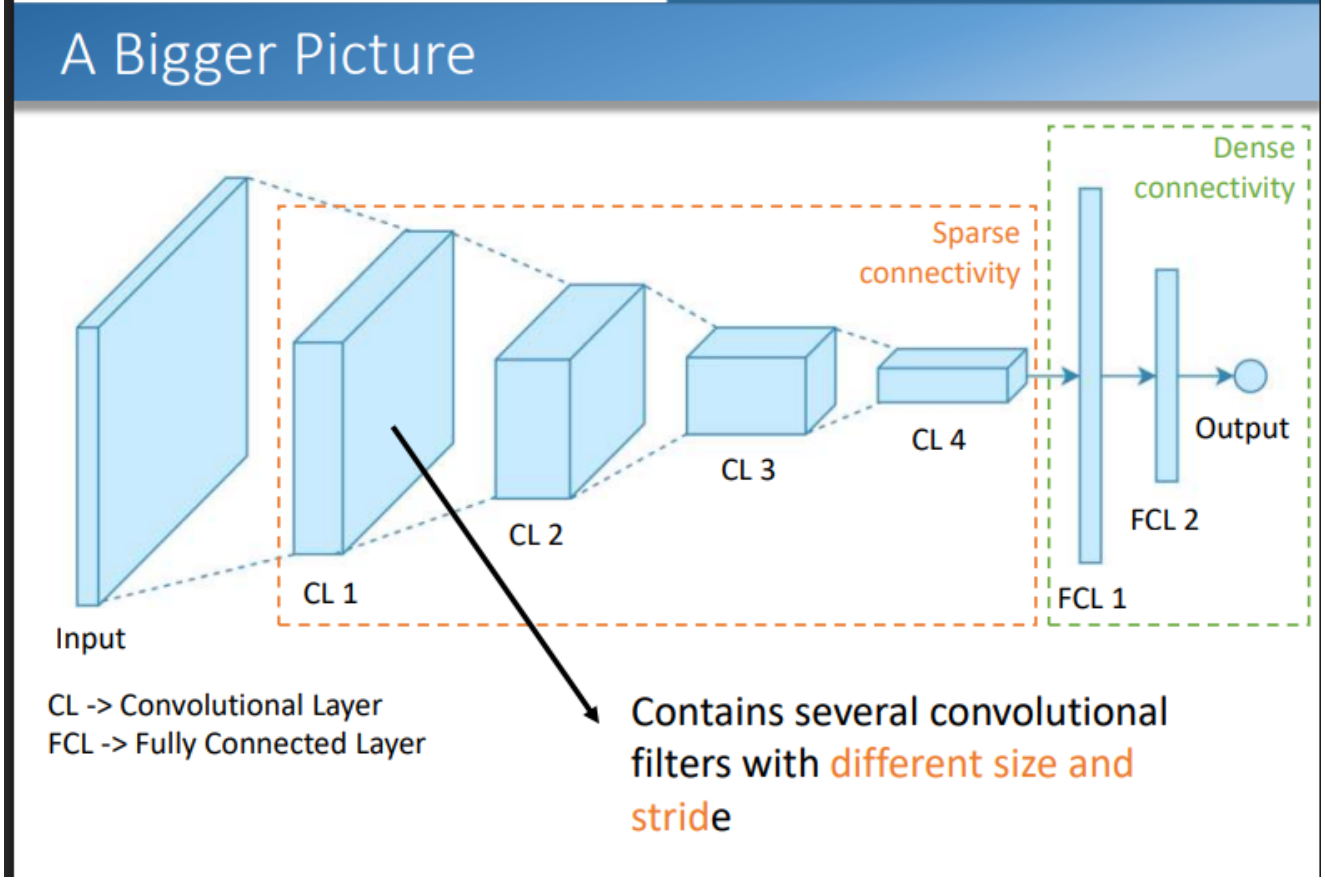


Figure: Bigger Picture

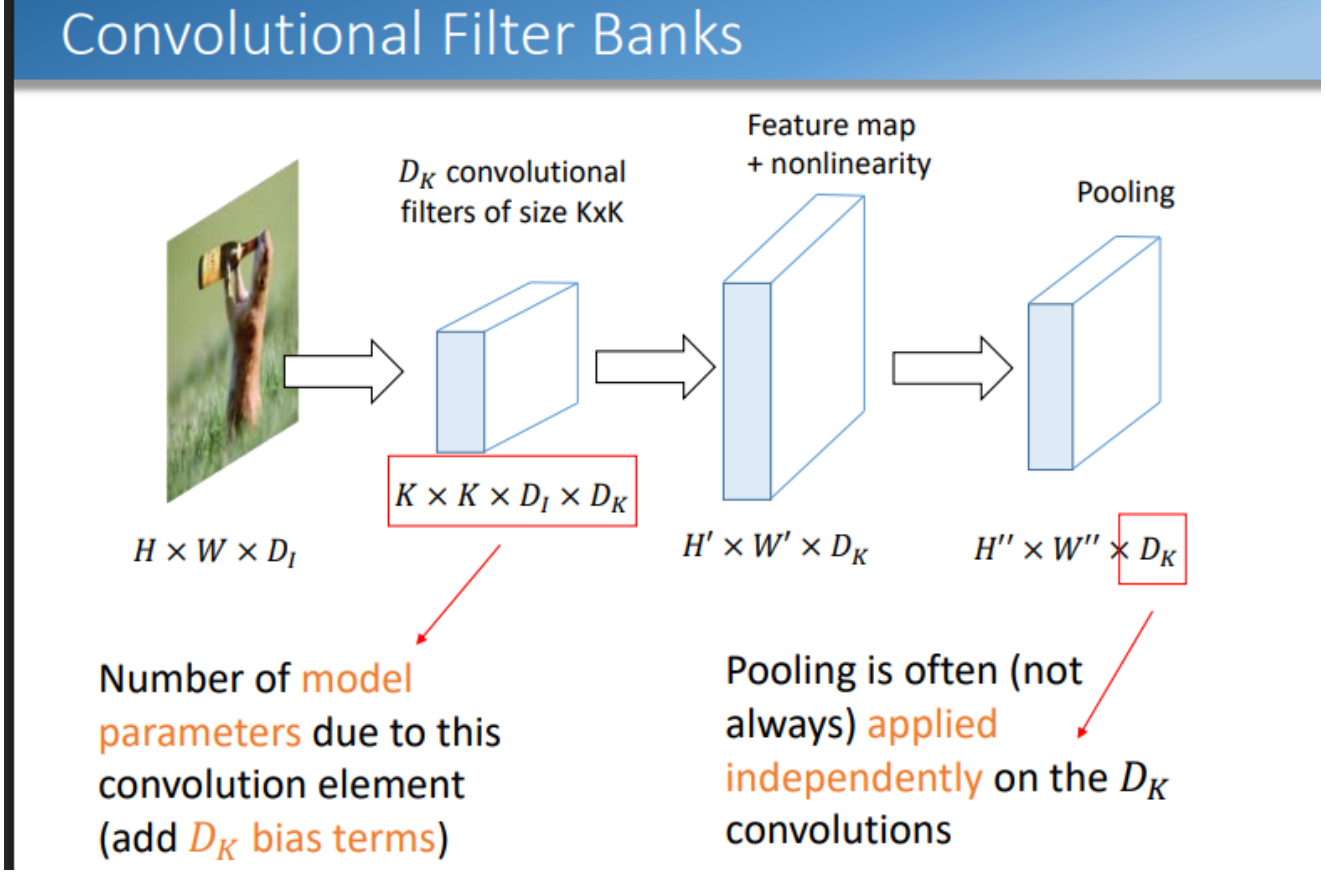


Figure: Convolutional Layers

CNN End-to-End Example

1. Input image: $32 \times 32 \times 3$
2. Conv layer: 32 filters $\rightarrow 32 \times 32 \times 32$
3. ReLU activation
4. Max pooling $\rightarrow 16 \times 16 \times 32$
5. Conv layer: 64 filters $\rightarrow 16 \times 16 \times 64$
6. Pooling $\rightarrow 8 \times 8 \times 64$
7. Flatten $\rightarrow$ vector of length 4096
8. Dense layer $\rightarrow$ 128 units
9. Softmax $\rightarrow$ 10 classes

Summary of CNN Steps

- Convolution extracts spatial patterns
- ReLU introduces non-linearity
- Padding preserves image size
- Stride controls downsampling
- Pooling reduces dimensionality
- Flatten converts features for Dense layers
- Fully connected layers classify

Specifying CNN in Code (Keras)

Number of convolution filters D_k

Define input size (only first hidden layer)

```
model = Sequential()
model.add(Conv2D(32, kernel_size=(5, 5), strides=(1, 1),
                 activation='relu',
                 input_shape=input_shape))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2, 2)))
model.add(Conv2D(64, (5, 5))
model.add(Activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(1000, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))
```

Does for you all the calculations to determine the final size to the dense layer (in most frameworks, you have to supply it)

Figure: Keras Example

Convolution: A Simple Example

Input (3x3):

$$\begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 1 \end{bmatrix}$$

Filter (2x2):

$$\begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

Output (2x2):

$$\begin{bmatrix} (1*1+2*0+0*(-1)+1*1) & \cdots \\ \cdots & \cdots \end{bmatrix}$$

Interpretation: Filter highlights spatial structure in the input.

Multi-Step Convolution on a 4×4 Image

Input image:

$$X = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 3 & 1 & 2 & 2 \\ 0 & 1 & 3 & 1 \\ 2 & 0 & 1 & 2 \end{bmatrix}$$

Filter (kernel):

$$K = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

Stride $= 1$, no padding. Output size:

$$(4 - 2 + 1) = 3 \Rightarrow 3 \times 3 \text{ output}$$

Step-by-Step Convolution (Top-Left Window)

Window:

$$\begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix}$$

Compute:

$$(1)(1) + (2)(0) + (3)(-1) + (1)(1)$$
$$= 1 + 0 - 3 + 1 = -1$$

Place at $(1, 1)$ of output.

Next Window (Top-Middle)

Window:

$$\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$$

Compute:

$$(2)(1) + (0)(0) + (1)(-1) + (2)(1)$$
$$= 2 + 0 - 1 + 2 = 3$$

Place at $(1, 2)$.

Next Window (Top-Right)

Window:

$$\begin{bmatrix} 0 & 1 \\ 2 & 2 \end{bmatrix}$$

Compute:

$$(0)(1) + (1)(0) + (2)(-1) + (2)(1)$$
$$= 0 + 0 - 2 + 2 = 0$$

Place at $(1, 3)$.

Final Output Feature Map

After scanning all windows:

$$Y = \begin{bmatrix} -1 & 3 & 0 \\ 2 & 1 & 3 \\ -3 & 2 & 1 \end{bmatrix}$$

Interpretation:

- Positive responses indicate matched patterns.
- Negative responses indicate opposing patterns.

Input and Filter

Input Image (5×5):

$$X = \begin{bmatrix} 1 & 2 & 0 & 1 & 3 \\ 3 & 1 & 2 & 2 & 0 \\ 1 & 0 & 1 & 3 & 1 \\ 2 & 1 & 0 & 1 & 2 \\ 0 & 3 & 1 & 2 & 1 \end{bmatrix}$$

Filter (3×3):

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 1 & 0 \\ 0 & -1 & 1 \end{bmatrix}$$

Stride $= 1$, Padding $= 0$. Output size:

$$(5 - 3 + 1) = 3 \quad \Rightarrow \quad 3 \times 3$$

Convolution Step: First Window

Window (top-left):

$$\begin{bmatrix} 1 & 2 & 0 \\ 3 & 1 & 2 \\ 1 & 0 & 1 \end{bmatrix}$$

Compute:

$$1(1) + 2(0) + 0(-1) + 3(1) + 1(1) + 2(0) + 1(0) + 0(-1) + 1(1) = 6$$

This becomes output element Y_{11}.

Complete Convolution Output

After computing all 9 windows:

$$Y = \begin{bmatrix} 6 & 6 & -1 \\ 1 & 1 & 7 \\ 1 & -1 & 0 \end{bmatrix}$$

Interpretation:

- Positive values $\rightarrow$ strong pattern match
- Negative values $\rightarrow$ opposite pattern

ReLU Activation

Apply ReLU:

$$R = \max(0, Y) = \begin{bmatrix} 6 & 6 & 0 \\ 1 & 1 & 7 \\ 1 & 0 & 0 \end{bmatrix}$$

Non-linear activation introduces deeper representational power.

Max Pooling (2×2, stride 1)

Pooling windows:

$$\begin{bmatrix} 6 & 6 \\ 1 & 1 \end{bmatrix} \Rightarrow 6 \qquad \begin{bmatrix} 6 & 0 \\ 1 & 7 \end{bmatrix} \Rightarrow 7$$

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} \Rightarrow 1 \qquad \begin{bmatrix} 1 & 7 \\ 0 & 0 \end{bmatrix} \Rightarrow 7$$

Final pooled map:

$$P = \begin{bmatrix} 6 & 7 \\ 1 & 7 \end{bmatrix}$$

Flatten Layer

Flatten row-wise:

$$\text{Flatten}(P) = [6,\ 7,\ 1,\ 7]$$

This vector can now enter a fully connected layer.

VALID vs SAME Padding

VALID padding (no padding):

$$\text{Output size} = n - k + 1$$

Example (5×5 input, 3×3 filter):

$$5 - 3 + 1 = 3 \Rightarrow 3 \times 3$$

SAME padding: Pad so output size = input size:

$$p = \frac{k-1}{2} \Rightarrow p = 1$$

Add 1-pixel border around the image:

$$X_{padded} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 0 & 1 & 3 & 0 \\ 0 & 3 & 1 & 2 & 2 & 0 & 0 \\ 0 & 1 & 0 & 1 & 3 & 1 & 0 \\ 0 & 2 & 1 & 0 & 1 & 2 & 0 \\ 0 & 0 & 3 & 1 & 2 & 1 & 0 \end{bmatrix}$$

Full Worked Example Summary

1. **Input:** 5×5 image
2. **Conv (3×3 filter):** produces 3×3 feature map

$$Y = \begin{bmatrix} 6 & 6 & -1 \\ 1 & 1 & 7 \\ 1 & -1 & 0 \end{bmatrix}$$

3. **ReLU applied:**

$$R = \begin{bmatrix} 6 & 6 & 0 \\ 1 & 1 & 7 \\ 1 & 0 & 0 \end{bmatrix}$$

4. **Max Pooling (2×2, stride 1):**

$$P = \begin{bmatrix} 6 & 7 \\ 1 & 7 \end{bmatrix}$$

5. **Flatten:** $[6, 7, 1, 7]$
6. **Ready for Dense Layer + Softmax**

Backprop Through Convolution: Overview

Goal: Compute gradients of loss L w.r.t.

$$W, \quad b, \quad X$$

Forward:

$$Y_{ij} = \sum_{u,v} X_{i+u,j+v} \, W_{uv} + b$$

Backward:

$$\frac{\partial L}{\partial W_{uv}} = \sum_{i,j} \frac{\partial L}{\partial Y_{ij}} X_{i+u,j+v}$$

$$\frac{\partial L}{\partial X_{ij}} = \sum_{u,v} \frac{\partial L}{\partial Y_{(i-u)(j-v)}} W_{uv}$$

Derivative w.r.t. Filter Weights

Given:

$$Y = X * W$$

Using chain rule:

$$\frac{\partial L}{\partial W_{uv}} = \sum_{i,j} \frac{\partial L}{\partial Y_{ij}} \frac{\partial Y_{ij}}{\partial W_{uv}}$$

Since:

$$\frac{\partial Y_{ij}}{\partial W_{uv}} = X_{i+u,j+v}$$

Thus:

$$\boxed{\frac{\partial L}{\partial W_{uv}} = \sum_{i,j} \delta_{ij} X_{i+u,j+v}}$$

Where:

$$\delta_{ij} = \frac{\partial L}{\partial Y_{ij}}$$

Derivative w.r.t. Input Feature Map

Using:

$$Y_{ij} = \sum_{u,v} X_{i+u,j+v} W_{uv}$$

Then:

$$\frac{\partial L}{\partial X_{pq}} = \sum_{i,j} \delta_{ij} \frac{\partial Y_{ij}}{\partial X_{pq}}$$

Non-zero when $(p, q) = (i + u, j + v)$:

$$\boxed{\frac{\partial L}{\partial X_{pq}} = \sum_{u,v} \delta_{(p-u)(q-v)} \, W_{uv}}$$

This is effectively a full convolution of δ with flipped filter.

Numerical Backprop Example

Suppose:

$$Y = X * W$$

and the gradient from next layer is:

$$\delta = \begin{bmatrix} 1 & -1 \\ 2 & 0 \end{bmatrix}$$

Let filter:

$$W = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}$$

Compute gradient w.r.t W:

$$\frac{\partial L}{\partial W} = X \star \delta$$

Each weight's gradient is a sum of products of overlapping patches.
Example for W_{11}:

$$\frac{\partial L}{\partial W_{11}} = \sum_{i,j} \delta_{ij} X_{i,j}$$

Conceptual Overview of CNN Backpropagation

Backpropagation in CNNs is an elegantly structured mathematical process that enables the network to learn discriminative visual features from high-dimensional image data. Its power emerges from the interplay of:

- **Calculus**: Chain rule for computing gradients.
- **Linear Algebra**: Efficient tensor computations.
- **Numerical Optimization**: SGD, Momentum, Adam.

The mechanism updates filter weights through:

- gradient flow from deep layers $\rightarrow$ shallow layers,
- accumulation of gradients over all positions where a shared filter is applied,
- propagation through pooling layers and nonlinear activations.

This allows CNNs to excel in fields such as autonomous driving, medical imaging, security, and environmental sensing.

Mathematics of Convolution Backpropagation

For a filter K producing output feature map O, the gradient is:

$$\frac{\partial L}{\partial K(m,n)} = \sum_i \sum_j \frac{\partial L}{\partial O(i,j)} I(i+m, j+n)$$

Interpretation:

- Each filter weight receives a gradient from every position where it is applied.
- This is **weight sharing**: the same filter scans across the whole image.
- Backprop "slides" over the input just like the forward convolution.

Convolutional Layer Forward Pass

Given input I, filter F, and bias b:

$$O = (I * F) + b$$

where $*$ denotes convolution.

Activation:

$$O = \sigma\left((I * F) + b\right)$$

For ReLU:

$$\sigma(x) = \max(0, x)$$

Pooling: (max or average pooling)
Reduces spatial dimensions while preserving important features.

Loss Calculation

For classification, the cross-entropy loss is:

$$L = -\sum_{c} y_c \log(\hat{y}_c)$$

where $\hat{y}_c$ = predicted probability of class c $y_c = 1$ if correct class, else 0.

Loss quantifies the error between prediction and ground truth.

Backward Pass: Overview

Goal: Compute gradients $\frac{\partial L}{\partial \theta}$ for all parameters θ.
Main gradient components:

1. Gradient w.r.t. output of last layer
2. Gradient through activation function
3. Gradient w.r.t. convolution filters
4. Gradient w.r.t. bias
5. Gradient through pooling layers

All use the **chain rule**:

$$\frac{\partial L}{\partial A} = \frac{\partial L}{\partial B} \cdot \frac{\partial B}{\partial A}$$

Gradient w.r.t. Output

For the final layer output O:

$$\frac{\partial L}{\partial O}$$

This depends on the loss function. Example (cross-entropy + softmax):

$$\frac{\partial L}{\partial O} = \hat{y} - y$$

Gradient Through Activation Function

$$\frac{\partial L}{\partial I} = \frac{\partial L}{\partial O} \cdot \frac{\partial O}{\partial I}$$

For ReLU:

$$\frac{\partial O}{\partial I} = \begin{cases} 1 & I > 0 \\ 0 & I \leq 0 \end{cases}$$

Interpretation: Only activated neurons propagate gradients.

Gradient w.r.t. Convolution Filters

For convolution output:

$$O = I * F$$

The gradient w.r.t. filter weights:

$$\frac{\partial L}{\partial F} = \frac{\partial L}{\partial O} * \text{rot}_{180}(I)$$

where $\text{rot}_{180}(I)$ rotates the input by 180°.

Why rotate?

- Backprop uses correlation, not convolution.
- Rotation aligns indices to compute correct gradients.

To backpropagate into the input I:

$$\frac{\partial L}{\partial I} = \frac{\partial L}{\partial O} * \text{rot}_{180}(F)$$

Interpretation: Input receives gradients from all filter positions that used it.

Gradient w.r.t. Biases

Bias gradient is a simple sum:

$$\frac{\partial L}{\partial b} = \sum_{i,j} \frac{\partial L}{\partial O(i,j)}$$

Each output unit contributes to the bias.

Backprop Through Pooling Layers

Max Pooling:

- Gradient goes ONLY to the maximum value in each window.

Average Pooling:

$$\frac{\partial L}{\partial I_{ij}} = \frac{1}{k^2} \frac{\partial L}{\partial O}$$

Pool downsampling reduces spatial resolution of gradients.

Parameter Update Rule

Using Stochastic Gradient Descent:

$$F_{\text{new}} = F_{\text{old}} - \eta \frac{\partial L}{\partial F}$$

$$b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial L}{\partial b}$$

- η = learning rate
- Similar updates apply to all parameters in CNN layers

Vanishing Gradients

In deep networks:

$$\left|\frac{\partial L}{\partial \theta}\right| \to 0$$

Causes:

- Sigmoid/tanh saturating regions
- Deep multiplication of small derivatives

Solutions:

- ReLU activation
- Batch normalization
- Residual connections

Exploding Gradients

Gradients become excessively large:

$$\left|\frac{\partial L}{\partial \theta}\right| \to \infty$$

Causes:

- Multiplication of large derivatives
- Unstable parameter initialization

Solutions:

- Gradient clipping
- Better weight initialization (He, Xavier)

Summary

Backpropagation in CNNs involves:

- Forward pass through convolution, activation, pooling
- Loss calculation
- Backprop through output, activation, convolution, pooling
- Updating filters and biases using SGD or Adam

Understanding the mathematics ensures correct model training and deeper insight into how CNNs learn complex visual patterns.

Practice Exercises- Q1: Manual Convolution Computation

Given the input:

$$I = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 1 & 1 \\ 0 & 2 & 2 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$$

with stride 1 and no padding, compute the output feature map.

Solution:

Patch (1,1):

$$1 \cdot 1 + 2 \cdot (-1) + 3 \cdot 0 + 1 \cdot 1 = 1 - 2 + 0 + 1 = 0$$

Patch (1,2):

$$2 \cdot 1 + 0 \cdot (-1) + 1 \cdot 0 + 1 \cdot 1 = 2 + 0 + 0 + 1 = 3$$

Patch (2,1):

$$3 \cdot 1 + 1 \cdot (-1) + 0 \cdot 0 + 2 \cdot 1 = 3 - 1 + 0 + 2 = 4$$

Patch (2,2):

$$1 \cdot 1 + 1 \cdot (-1) + 2 \cdot 0 + 2 \cdot 1 = 1 - 1 + 0 + 2 = 2$$

$$O = \begin{bmatrix} 0 & 3 \end{bmatrix}$$

Q2: Output Size With Stride and Padding

For an input of size 7×7, filter size 3×3, stride $s = 2$, padding $p = 1$:

$$\text{Output size} = \frac{N - F + 2p}{s} + 1$$

Solution:

$$\frac{7 - 3 + 2(1)}{2} + 1 = \frac{6}{2} + 1 = 3 + 1 = 4$$

So output is 4×4.

Q3: Sliding Window Interpretation

Explain in your own words what the sliding window (filter movement) accomplishes in a CNN.

Answer:

The sliding window allows filters to scan across the entire image, detecting patterns in every region. Instead of assigning a separate weight per pixel (like an MLP), CNNs reuse the same filter everywhere. This gives CNNs:

- translation invariance
- local pattern detection
- lower number of parameters

Q4: ReLU After Convolution

Given a convolution output:

$$O = \begin{bmatrix} -4 & 2 \\ 5 & -1 \end{bmatrix}$$

Apply ReLU activation.
Solution:

$$O_{\text{ReLU}} = \begin{bmatrix} 0 & 2 \\ 5 & 0 \end{bmatrix}$$

Q5: Parameter Count in a Conv Layer

How many learnable parameters are in a convolutional layer with:

- 32 filters
- filter size 3×3
- input depth = 3 (RGB)

Solution:
Each filter has:

$$3 \times 3 \times 3 = 27 \text{ weights}$$

plus a bias:

$$27 + 1 = 28$$

Total parameters:

$$32 \times 28 = 896$$

Much fewer than a fully connected layer.

Q6: CNN vs MLP on Images

Why does a CNN outperform an MLP on image classification?
Answer:

- CNNs use weight sharing $\rightarrow$ far fewer parameters.
- CNNs preserve spatial structure; MLPs flatten images.
- CNNs detect local features such as edges, textures, shapes.
- CNNs are translation invariant.

Q7: Pooling Operation

Given:

$$\begin{bmatrix} 4 & 1 \\ 7 & 3 \end{bmatrix} \quad \begin{bmatrix} 5 & 2 \\ 0 & 8 \end{bmatrix}$$

Apply 2×2 max pooling over each 2×2 block.

Solution:

$$\text{max pool} = \begin{bmatrix} 7 & 8 \end{bmatrix}$$

Q8: Compute Output Size (General Case)

Given:

$$N = 28, \quad F = 5, \quad p = 2, \quad s = 1$$

Solution:

$$\frac{28 - 5 + 2(2)}{1} + 1 = 28$$

Output remains 28×28. This is why padding is sometimes called **same padding**.

Q9: Stride 2 Example

Input width = 10, filter size = 3, stride = 2, padding = 0.

$$\frac{10 - 3 + 0}{2} + 1 = \frac{7}{2} + 1 = 3.5 + 1$$

Since output size must be integer:

$$\Rightarrow 4 \text{ (floor)}$$

Q10: Why Backprop Through CNN is Efficient

Explain briefly why backpropagation through a CNN is computationally efficient.

Answer:

- Filters are shared $\rightarrow$ gradients also shared $\rightarrow$ fewer parameters.
- Convolutions are structured $\rightarrow$ can reuse computations.
- GPUs accelerate convolution massively due to parallelism.

Q11: Gradient Descent Update Rule

Write the gradient descent update rule for a weight w with learning rate η.

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

Concept: Move weight opposite the direction of steepest ascent of the loss.

Q12: Learning Rate Too Large

What happens if the learning rate is too large?
Answer:

- The loss oscillates or diverges.
- The model fails to converge.
- Training becomes unstable.

Q13: Learning Rate Too Small

What happens if the learning rate is too small?
Answer:

- Very slow convergence.
- Model may get stuck in local minima or ravines.
- Training time becomes extremely long.

Q14: Multiple Filters Output Depth

If the input is $28 \times 28 \times 3$ and the convolution layer uses 64 filters, what is the depth of the output volume?

Answer: The depth equals the number of filters:

$$\text{depth} = 64$$

Q15: Flatten Layer Size

If you have a feature map of shape $16 \times 16 \times 32$, what is the length of the flattened vector?

$$16 \cdot 16 \cdot 32 = 8192$$

Answer: 8192.

Q16: Why CNNs Learn Hierarchical Features

Explain why deeper CNN layers learn more abstract patterns.
Answer:

- Early layers learn edges and simple textures.
- Middle layers learn shapes and object parts.
- Deep layers learn high-level concepts (faces, digits, etc.).

This hierarchical composition is natural for images.

Q17: Purpose of Padding

Why is padding used?
Answer:

- Prevent shrinking of output feature maps.
- Preserve spatial information near boundaries.
- Control output size.

Q18: Mini-Batch Gradient Descent in CNNs

Why do CNNs usually use mini-batch gradient descent?
Answer:

- More efficient than full batch.
- More stable than pure SGD.
- Allows parallelization on GPUs.

Q19: Numerical Backprop Example

Given:

$$w = 2, \quad \frac{\partial L}{\partial w} = 0.4, \quad \eta = 0.1$$

Compute the new weight:

$$w_{\text{new}} = 2 - 0.1(0.4) = 2 - 0.04 = 1.96$$

Answer: 1.96.

Q20: Interpretation of Feature Maps

Explain what a feature map represents.

Answer: A feature map stores the activation responses of a learned filter at different locations in the input. Bright regions mean the filter strongly detected its pattern (edge, curve, texture).

Key CNN Backprop Insights

- Gradients w.r.t filters are computed by convolving the input with the upstream gradient.
- Gradients w.r.t input are computed by convolving the gradient with the flipped filter.
- Weight sharing implies gradients accumulate across all spatial locations.
- This makes CNNs efficient and powerful for images.

Further Practice Exercises

1. Compute gradients for a 2×2 filter given a 3×3 input and loss gradients.
2. Explain weight sharing and how it reduces parameter count.
3. Derive gradients for max pooling for a given 2×2 region.
4. Modify the CNN to include BatchNorm and Dropout.
5. Visualize the learned filters of the first layer.

Summary

- CNNs extract spatial features using convolution + pooling.
- Backpropagation in CNNs involves structured gradient flow:
  - shared filter updates,
  - backprop through pooling,
  - backprop through activations.
- These mathematical principles enable CNNs to perform exceptionally well in vision tasks.

References

Fleuret, F. (2024).

The Little Book of Deep Learning.

University of Geneva.

Goodfellow, I., Bengio, Y., Courville, A. (2016).

Deep Learning.

MIT Press, Cambridge.

Bishop, C. M., & Bishop, H. (2023).

Deep Learning: Foundations and Concepts.

Springer Nature.

Prince, S. J. (2023).

Understanding Deep Learning.

MIT Press.